Project Proposal: CNN vs. Classical Machine Learning vs. Audio Foundation Models for Musical Instrument Recognition

Team Neural Band: Aisha Abouganba (aa2381), Bhagyashri Farkade (bbf24), Srija Dwarakacherla (sd2003)

1. Research Questions

RQ1. Can a deep learning model (CNN) that looks at pictures of sound (spectrograms) recognize the main instrument in a music clip better than a simple machine learning model that uses hand-made sound features?

- *Positive answer:* The CNN scores at least 5 points higher on macro-F1 than the simple model, consistently across 3 training runs.
- *Negative answer:* The CNN is not clearly better, showing that with a small dataset deep learning is not always worth the extra effort.

RQ2. How do large pretrained audio foundation models (AST, PANNs, CLAP) compare to our CNN trained from scratch, in a **zero-shot** setting (no training on IRMAS) and a **fine-tuned** setting, especially when labeled data is scarce?

- *Positive answer:* The best fine-tuned foundation model beats our CNN by at least 5 macro-F1 points across 3 runs, and its advantage is largest at the smallest training-set sizes; zero-shot models clearly beat the majority-class baseline.
- *Negative answer:* A small CNN trained from scratch matches the foundation models, suggesting large-scale pretraining adds little for this task.

2. Motivation and Prior Work

Recognizing instruments helps with music search, tagging, recommendation, and learning apps. It is hard in real songs because many instruments play at the same time. Eronen & Klapuri (2000) showed MFCC features can tell instruments apart; our simple model uses this idea. Bosch et al. (2012) created the IRMAS dataset of real music clips, which we will use. Han, Kim & Lee (2017) showed a CNN on spectrograms works well on IRMAS; our CNN follows their approach. Park et al. (2019) introduced SpecAugment, which makes models more robust by masking parts of the spectrogram during training.

Today, many audio models are pretrained on massive datasets and can be loaded and run directly. PANNs (Kong et al., 2020) are CNNs and AST (Gong et al., 2021) is a transformer, both trained on AudioSet (Gemmeke et al., 2017), whose label set already includes most IRMAS instruments. CLAP (Elizalde et al., 2023) learns a shared space for audio and text, so it can classify a clip by comparing it to text prompts such as “the sound of a violin.”

What we add: a fair side-by-side comparison of three generations of approaches (hand-crafted features, a CNN from scratch, and pretrained foundation models) on the same data with a careful split, a study of how each behaves as training data shrinks, and tests showing which design choices actually help.

3. Data, Compute, and Method

- **Data:** IRMAS training set, about 6,700 three-second clips of 11 instruments (piano, violin, guitar, flute, and others). Free online. Access: [Yes / Not yet]. Pretrained weights for AST, PANNs, and CLAP are publicly available. **Compute:** Our laptops, or free Google Colab GPUs for CNN training and foundation-model fine-tuning. Access: [Yes].
- **Classical and CNN models:** We clean all audio (same sample rate, remove silence, equal volume) and cut it into 3-second pieces. The simple model is a Random Forest or SVM using MFCC and other sound features; the deep model is a small CNN trained on mel spectrograms.
- **Foundation models, zero-shot:** No training on IRMAS. For AST and PANNs, we map their AudioSet labels to the 11 IRMAS classes (e.g., “Violin, fiddle” $\rightarrow$ violin) and predict the class with the highest score. For CLAP, we compare each clip to one text prompt per instrument and pick the closest.
- **Foundation models, fine-tuned:** We (a) train a simple classifier on frozen embeddings (linear probe) and (b) fine-tune the full model (or its top layers, if compute is limited) on IRMAS, replacing the output layer with our 11 classes.

- **Data-scarcity study:** We train the simple model, the CNN, and the fine-tuned foundation models on 5%, 10%, 25%, 50%, and 100% of the training data (class-balanced subsets, same recordings-aware split) and plot macro-F1 against training-set size.
- **Tools:** Python, librosa, scikit-learn, PyTorch, Hugging Face Transformers (AST, CLAP), and the official PANNs code.

4. Evaluation Plan

- **Metrics:** Macro-F1 (main metric, fair to rare instruments), accuracy, and a confusion matrix to see which instruments get mixed up. We also report model size and training time to weigh accuracy against cost.
- **Baselines:** an “always guess the most common instrument” model, the simple model, our CNN, and the zero-shot foundation models.
- **Ablations:** CNN with and without SpecAugment, clip length (1, 3, or 5 seconds), random split vs. careful split, linear probe vs. full fine-tuning, and amount of training data (data-scarcity curves).
- **Avoiding data leakage:** Pieces cut from the same recording always stay in the same set (train, validation, or test). We choose settings using only the validation set and use the test set once at the end. All models use the exact same splits and subsets.

5. Risks and Fallback

The most likely problem is that the CNN memorizes the training data (overfitting) and does not beat the simple model. If so, we will add more augmentation and use a smaller model. If the CNN still does not win, we will report this honest negative result with our analysis. For the foundation models, the main risks are (1) limited GPU time for full fine-tuning, in which case we will fine-tune only the top layers or rely on the linear probe, and (2) imperfect label mapping in zero-shot mode (e.g., several AudioSet labels such as “Organ” and “Hammond organ” map to one IRMAS class), which we will document and test with a few prompt or mapping variants chosen on the validation set.

6. Plan and Roles

Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
Setup, download data, read papers, explore data	Clean audio, extract features, split data	Train simple models; run zero-shot foundation models	Train CNN; extract pretrained embeddings	Fine-tune foundation models; linear probes	Ablations and data-scarcity experiments	Final testing, error analysis, demo app	Write report and presentation

- **Roles:** Aisha handles data, features, and the SVM/Random Forest; Bhagyashri implements and trains the CNN and leads foundation-model fine-tuning; Srija runs zero-shot evaluation, ablations, the data-scarcity study, and evaluation. All three review papers, analyze errors, and write the report.
- **Use of LLMs:** We used Claude to brainstorm the project plan, clarify concepts, and help draft and revise parts of the proposal and code. We will verify its suggestions, read the cited papers ourselves, run the experiments, and check all results. We may use it for debugging and writing feedback, and will disclose any substantial use in the final report.

References

[1] Eronen, A., & Klapuri, A. (2000). Musical instrument recognition using cepstral coefficients and temporal features. ICASSP.

[2] Bosch, J. J., Janer, J., Fuhrmann, F., & Herrera, P. (2012). A comparison of sound segregation techniques for predominant instrument recognition in musical audio signals. ISMIR.

[3] Han, Y., Kim, J., & Lee, K. (2017). Deep convolutional neural networks for predominant instrument recognition in polyphonic music. IEEE/ACM TASLP.

[4] Park, D. S., et al. (2019). SpecAugment: A simple data augmentation method for automatic speech recognition. Interspeech.

[5] Gemmeke, J. F., et al. (2017). Audio Set: An ontology and human-labeled dataset for audio events. ICASSP.

[6] Kong, Q., Cao, Y., Iqbal, T., Wang, Y., Wang, W., & Plumbley, M. D. (2020). PANNs: Large-scale pretrained audio neural networks for audio pattern recognition. IEEE/ACM TASLP.

[7] Gong, Y., Chung, Y.-A., & Glass, J. (2021). AST: Audio Spectrogram Transformer. Interspeech.

[8] Elizalde, B., Deshmukh, S., Al Ismail, M., & Wang, H. (2023). CLAP: Learning audio concepts from natural language supervision. ICASSP.